Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Laboratory Activity No. 2
Personal MP3 Player
Laboratory Report

Development Environment: PlatformIO, STM32CubeF4, FreeRTOS, RT-Spark / STM32F407ZG
Repository: github.com/msyennna/stm32-freertos-audio-player
Date: October 14, 2026

Phoebien D. Cañete

Shanice Reih H. Tanque

1. Problem and Requirements

The project implements a personal audio player on an STM32F407ZG-based RT-Spark platform using FreeRTOS. The firmware integrates binary song selection, synthesized melody playback, an onboard LCD, an ES8388 audio codec, potentiometer volume control, RGB state indicators, UART instructions, and pause/resume control. The laboratory requirement is addressed with three application tasks while audio streaming remains interrupt- and DMA-driven. The player exposes eight songs through a 3-bit selection made with B2-B4. B1 is used first to capture the binary choice and again to confirm it. A FreeRTOS one-shot timer limits the confirmation window to five seconds. The built-in UP button provides pause/resume control, while a potentiometer sets the playback volume. The implementation synthesizes PCM audio from note and beat definitions. It does not decode stored MP3 files, even though the laboratory activity is titled Personal MP3 Player.

2. System Architecture and Design

2.1 Hardware Architecture

Component / Signal	Connection	Function
B1	PE2	Capture / confirm song selection
B2	PE3	Selection bit 0
B3	PE4	Selection bit 1
B4	PE5	Selection bit 2
Built-in UP	PC5	Pause / resume, active-low
Potentiometer	PA0 / ADC1 CH0	12-bit volume input
RGB LEDs	PF5 / PF6 / PF7	Red / green / blue state indication
ES8388 control	PF0 SDA / PF1 SCL, I2C2	Codec initialization and control
I2S3 audio	PA15, PB3, PB5, PC7	WS, CK, SD, MCLK
UART1	PA9 / PA10	115200-baud instructions and serial output

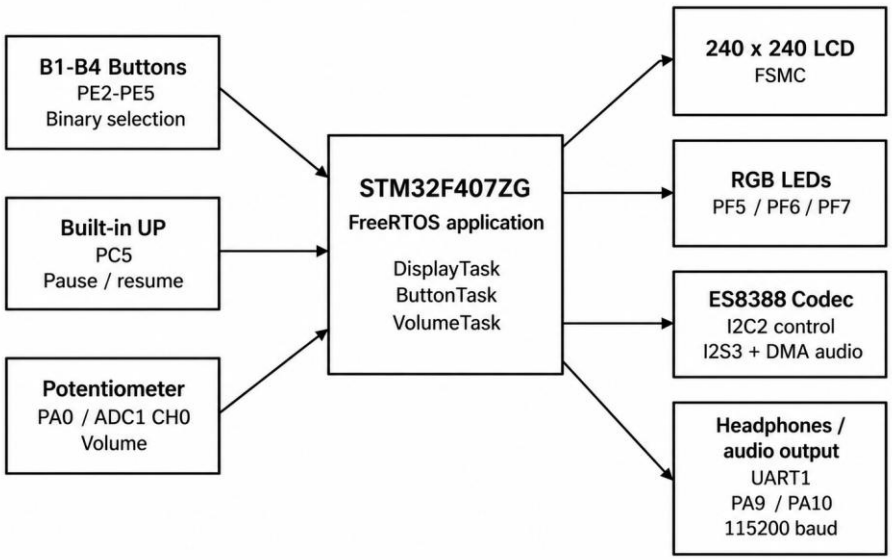


Figure 1. Hardware and peripheral architecture of the FreeRTOS audio player.

2.2 Song Selection Mapping

Binary (B4 B3 B2)	Song	Source label
000	1	Fur Elise - Beethoven
001	2	Canon In D - Pachebelbel
010	3	Minuet in G major - Bach
011	4	Turkish March - Mozart
100	5	Nocturne in E flat - Chopin
101	6	Waltz No. 2 - Shostakovich
110	7	Nocturne in C sharp - Chopin
111	8	Symphony No. 40 - Mozart

The source table contains ten melodies, but the selection expression `(stable >> 1) & 7U` limits the external selector to the first eight entries. B2 is the least-significant song bit and B4 is the most-significant bit.

3. FreeRTOS Architecture

Task	Stack	Priority	Primary responsibility
DisplayTask / StartDisplayTask	512 words	1	LCD status, RGB LEDs, UART startup instructions, and starting confirmed songs
ButtonTask / StartButtonTask	256 words	1	40 ms debounced B1-B4 input and built-in UP pause/resume
VolumeTask / StartVolumeTask	256 words	1	ADC1 sampling, smoothing, percentage mapping, and audio volume update

All three application tasks are created with priority 1. DisplayTask and VolumeTask delay for 20 ms per loop, while ButtonTask delays for 10 ms. The design therefore relies on frequent blocking delays to share CPU time among equal-priority tasks.

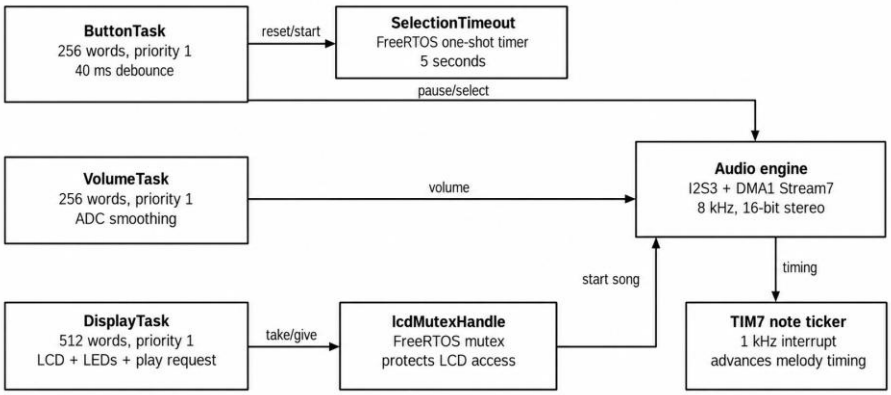


Figure 2. FreeRTOS task, timer, mutex, and interrupt/DMA relationships.

3.1 Synchronization and Timing

- A FreeRTOS mutex (`lcdMutexHandle`) is created to serialize LCD access.
- A one-shot software timer named `SelectionTimeout` is configured for 5000 ms.
- Button changes are accepted only after 40 ms of stable input.
- TIM7 generates a 1 kHz interrupt used to advance melody note timing.
- DMA1 Stream7 runs in circular mode and refills audio buffer halves through I2S callbacks.
- The FreeRTOS idle hook enters light sleep using `HAL_PWR_EnterSLEEPMode(..., WFI)`.

4. Implementation

4.1 Song Capture and Confirmation

The PE2-PE5 input register is inverted because the external buttons are active-low. When B1 is pressed and no selection is active, the firmware captures `(stable >> 1) & 7U` into `pendingSong`, records the current tick count, marks the selection active, and resets the five-second timer. A later B1 press while the selection remains active copies the pending song into `currentSong`, stops the timer, and increments `playRequest`. DisplayTask detects the changed request counter and calls `SparkAudio_PlaySong()` for the confirmed selection.

4.2 Audio Generation and Codec Output

The audio engine reads song metadata through `SparkSongs_Get()`. Each melody consists of note periods, beat values, a tempo value, and a length. `SparkAudio_PlaySong()` converts this information into phase increments and note durations. Audio samples are generated as a triangle waveform with a short attack and release envelope to reduce clicking between notes.

I2S3 is configured as master transmit using Philips I2S, 16-bit data, master clock output, and an 8 kHz audio frequency. DMA1 Stream7 Channel 0 transfers a circular 128-halfword buffer. The ES8388 codec is controlled through I2C2 at 100 kHz using the 7-bit address 0x10.

4.3 Potentiometer Volume Control

PA0 is configured as ADC1 Channel 0 with 12-bit right-aligned conversion. VolumeTask filters the raw reading using `(filteredRaw * 7 + raw + 4) / 8`, maps the result to 0-100%, snaps values at 1% or below to 0% and 99% or above to 100%, and ignores changes smaller than two percentage points. Accepted changes are passed to `SparkAudio_SetVolume()`, which changes the generated PCM amplitude.

4.4 LCD, LEDs, and UART

The active PlatformIO build uses `AudioPlayer/Core/Src/main.c`. Its display task renders a text-based status screen containing codec detection, song number/title, playback state, volume, and pending choice. RGB outputs use PF5 as red, PF6 as green, and PF7 as blue. Blue indicates playing, green indicates an active selection window, and red is enabled when the player is not running and no selection is active.

UART1 is configured for 115200 baud on PA9/PA10. At startup, the firmware prints instructions explaining the B1-B4 binary selection sequence, the five-second confirmation step, the built-in UP pause/resume control, and the potentiometer volume control.

4.5 Repository Configuration Note

The repository contains `AudioPlayer/Core/Src/main.cpp`, which implements the newer graphical SPARK PLAYER interface with progress information, waveform animation, a modern player card, and a confirmation countdown. The current `platformio.ini` source filter still selects `Core/Src/main.c`. However, the supplied physical-board photograph shows the graphical interface on the RT-Spark LCD, including the SPARK PLAYER header, song information, volume bar, and FINISHED state. This indicates that the graphical variant was flashed and tested on the hardware at some point. For reproducibility, the repository configuration should be aligned with the exact source file used for the photographed build.

5. Verification and Testing

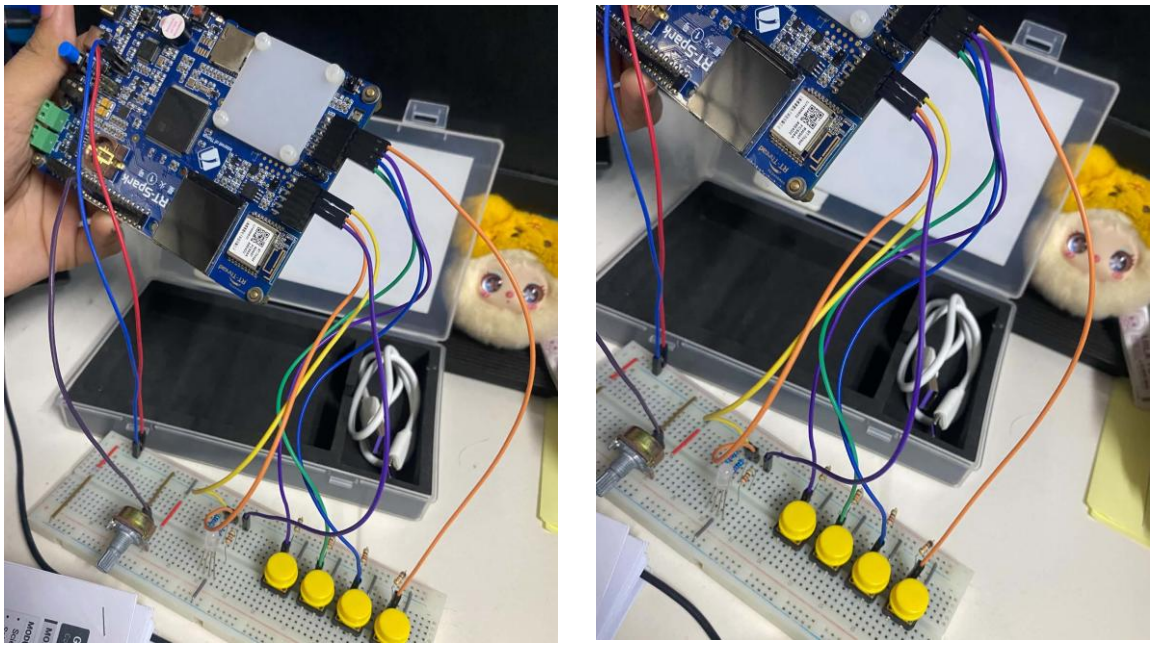
No application-level automated tests are present in the repository; the `test/` folder contains only the default PlatformIO placeholder README. Physical photographs supplied for this report provide evidence of the assembled RT-Spark prototype, external buttons, potentiometer, RGB LED hardware, and a graphical LCD state. Still photographs cannot prove timing-dependent behavior such as the five-second timeout, pause/resume transitions, all eight selections, or continuous audio quality, so those items remain marked as not captured.

ID	Test	Expected result	Status / evidence
FT-01	Select 000 and confirm	Song 1 starts and LCD updates	Photo evidence: LCD shows 01/08, Fur Elise, FINISHED
FT-02	Repeat 001 through 111	Correct first eight songs are selected	Not captured in supplied photos
FT-03	Do not confirm within 5 s	Pending selection cancels	Not captured in supplied photos
FT-04	Press built-in UP during playback	Playback toggles pause/resume	Not captured in supplied photos
FT-05	Rotate potentiometer	Volume changes smoothly with limited jitter	Potentiometer present; LCD shows 70% volume (movement not captured)
FT-06	Observe RGB states	Blue=playing, green=selecting, red=not playing	Red LED observed at FINISHED; blue/green states not captured
FT-07	Open serial monitor at 115200	Startup instructions are printed	Not captured - no serial-monitor screenshot supplied
FT-08	Run full song playback	Audio continues without DMA/I2S error	LCD shows FINISHED; continuous audio quality not captured

The photographs below are used as physical evidence of the implemented prototype. They document the RT-Spark board, external B1-B4 controls, potentiometer, wiring, RGB LED hardware, and the graphical player screen. They are not used to claim behavior that is not visible in a still image.

5.1 Physical Hardware Evidence

Figures 3 and 4 show the hardware prototype supplied for documentation. The images are included as observed evidence rather than reconstructed diagrams.



(a) Complete RT-Spark prototype with breadboard controls.

(b) External button, potentiometer, LED, and jumper-wire connections.

Figure 3. Physical prototype and external control wiring.

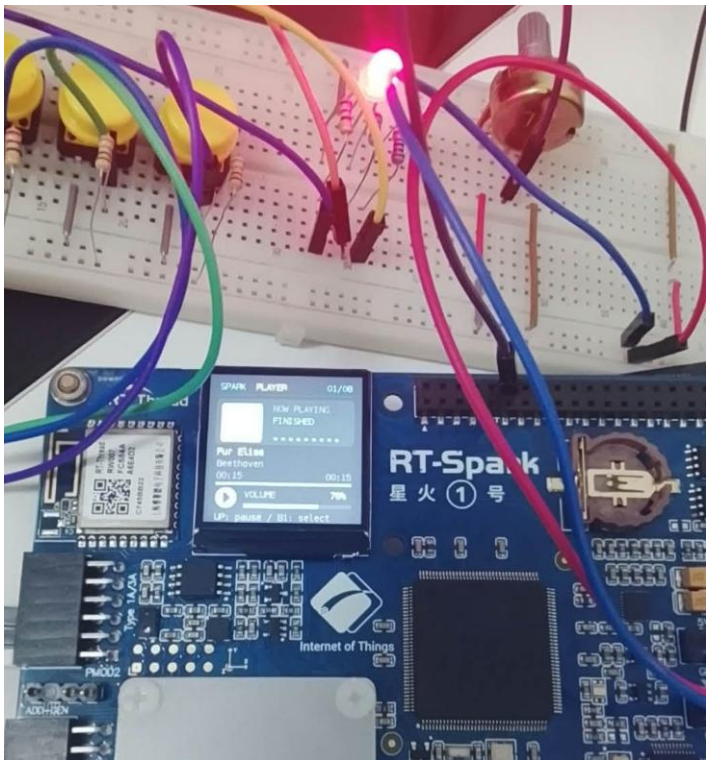


Figure 4. RT-Spark graphical player screen showing Fur Elise (01/08), FINISHED state, 70% volume, and a lit red status LED.

The graphical screen in Figure 4 corresponds to the newer player-interface implementation stored in `main.cpp`. This physical evidence is why the repository configuration note in Section 4.5 is retained: the checked-in PlatformIO filter and the photographed firmware should be made consistent before archival or reproduction.

6. Engineering Discussion

6.1 Design Strengths

- The three-task structure separates input, volume control, and presentation responsibilities.
- I2S DMA and TIM7 handle continuous audio timing outside the task loops.
- A one-shot FreeRTOS timer directly implements the five-second confirmation window.
- ADC smoothing, a 2% threshold, and an LCD mutex reduce jitter and resource conflicts.

6.2 Limitations and Documentation Risks

- The project synthesizes melodies and does not decode MP3 files; the 3-bit selector exposes eight of ten stored melodies.
- All three application tasks use priority 1, so responsiveness depends on short work loops and delays.
- The active PlatformIO filter builds `main.c`; the newer graphical UI is stored separately in `main.cpp`.
- Physical photographs are available and included in this report, but they do not document every timing-dependent functional test. UART output and live observations for all eight selections, timeout behavior, pause/resume, and LED transitions should be retained separately if required by the instructor.

7. Conclusion

The repository implements a FreeRTOS-based STM32F407ZG audio player that combines binary song selection, synthesized I2S/DMA audio, ES8388 codec control, ADC-based volume adjustment, LCD feedback, RGB status indication, UART instructions, and pause/resume control. The software architecture meets the three-task design by assigning presentation, button handling, and volume control to separate FreeRTOS tasks while keeping continuous audio generation in interrupt- and DMA-driven code. The supplied hardware photographs confirm that the project was assembled on the RT-Spark platform with the external controls and graphical LCD interface. For reproducibility, the pair should align `platformio.ini` with the exact application source used for the photographed build and retain serial or live-test evidence for behaviors that cannot be demonstrated by still images.